

LAB # 06 (A)

Exercise: Select your previous project and Design Test cases for at least 5 components using any of white box strategy

White Box Testing Report: Banking System

1. Functionality: Registration

Code :

```
def register():
```

```
    username = reg_username_entry.get().strip()
    password = reg_password_entry.get().strip()
    account_number = reg_account_number_entry.get().strip()
    pin = reg_pin_entry.get().strip()
```

B1: if not username or not password or not account_number or not pin:

```
    messagebox.showerror("Error", "All fields are required!")
```

```
    return
```

B2: if len(password) < 8:

```
    messagebox.showerror("Error", "Password must be at least 8 characters long!")
```

```
    return
```

B3: if not username.isalnum():

```
    messagebox.showerror("Error", "Username must contain only letters and numbers!")
```

```
    return
```

B4: if username in users:

```
    messagebox.showerror("Error", "User already exists!")
```

```
    return
```

B5: if not account_number.isdigit() or len(account_number) != 10:

```
    messagebox.showerror("Error", "Account number must be a 10-digit number!")
```

```
    return
```

B6: if len(pin) != 4 or not pin.isdigit():

```
    messagebox.showerror("Error", "PIN must be a 4-digit number!")
```

```
    return
```

```
    B7: users[username] = {"password": password, "account_number": account_number, "pin": pin,
    "balance": 0}
```

```
    messagebox.showinfo("Success", "Registration Successful!")
```

```
    reg_username_entry.delete(0, tk.END)
```

```
    reg_password_entry.delete(0, tk.END)
```

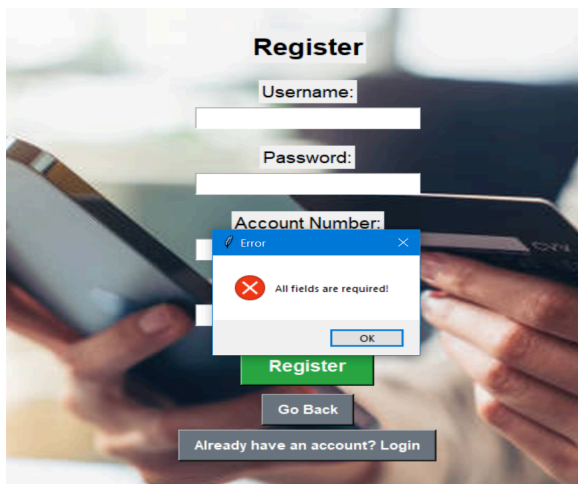
```
reg_account_number_entry.delete(0, tk.END)
reg_pin_entry.delete(0, tk.END)
```

Summary Report :

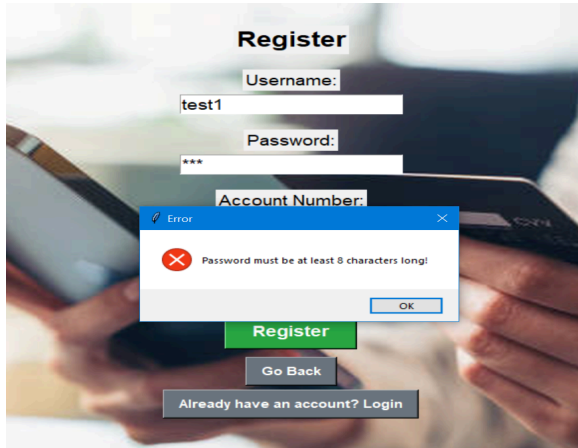
Test Case ID	Test Case Description	Test Steps	Test Data	Expected Result	Actual Result	Status	Branch Covered
TC_R01	Empty Username Field	1. Go to Register Page	Username: (empty),	Error: All fields are required	Error: All fields are required	Pass	B1
		2. Leave username empty	Password: pass1234,				
		3. Enter valid data in other fields	Account: 1234567890,				
		4. Click Register	PIN: 1234				
TC_R02	Short Password	1. Go to Register Page	Username: user1,	Error: Password must be at least 8 chars	Error: Password must be at least 8 chars	Pass	B2
		2. Enter valid username	Password: 123,				
		3. Enter short password	Account: 1234567890,				
		4. Click Register	PIN: 1234				
TC_R03	Invalid Username Characters	1. Go to Register Page	Username: user@1,	Error: Username must be alphanumeric	Error: Username must be alphanumeric	Pass	B3
		2. Enter username with symbols	Password: pass1234,				
		3. Fill other fields	Account: 1234567890,				
		4. Click Register	PIN: 1234				
TC_R04	Username Already Exists	1. Go to Register Page	Username: user1,	Error: User already exists	Error: User already exists	Pass	B4
		2. Enter existing username	Password: pass1234,				
		3. Fill valid fields	Account: 1234567890,				
		4. Click Register	PIN: 1234				
TC_R05	Invalid Account Number	1. Go to Register Page	Account Number: 12345	Error: Account number must be 10-digits	Error: Account number must be 10-digits	Pass	B5
		2. Enter less than 10 digits in account number					
		3. Click Register					
TC_R06	Invalid PIN	1. Go to Register Page	PIN: 123	Error: PIN must be 4-digit number	Error: PIN must be 4-digit number	Pass	B6
		2. Enter 3-digit PIN					
		3. Click Register					
TC_R07	Valid Registration	1. Go to Register Page	Username: newuser,	Registration successful	Registration successful	Pass	B7
		2. Enter valid details	Password: pass1234,				
		3. Click Register	Account: 1234567890,				
			PIN: 1234				

Test Cases

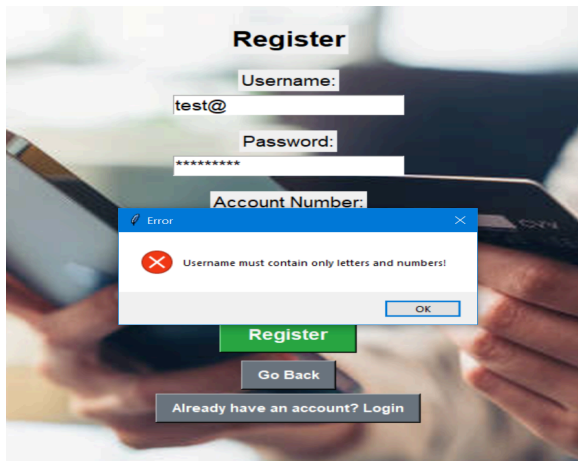
B1:



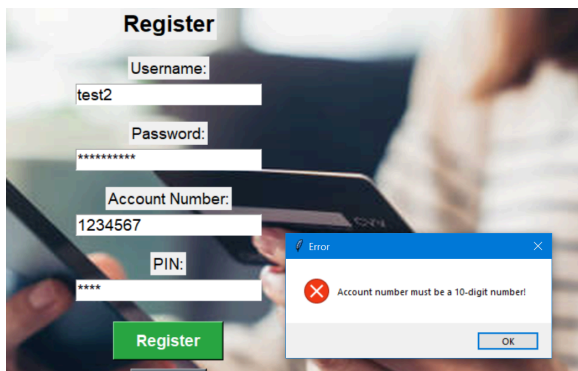
B2:



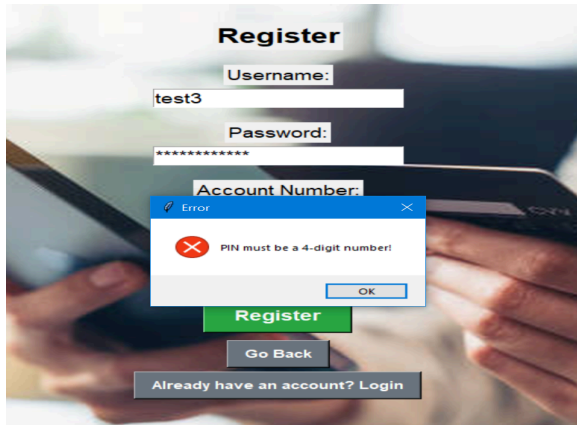
B3:



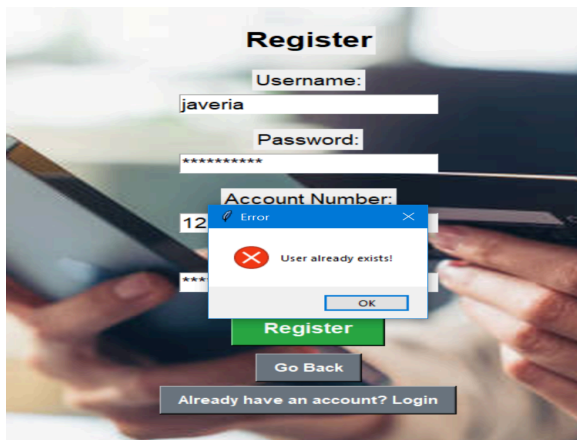
B5:



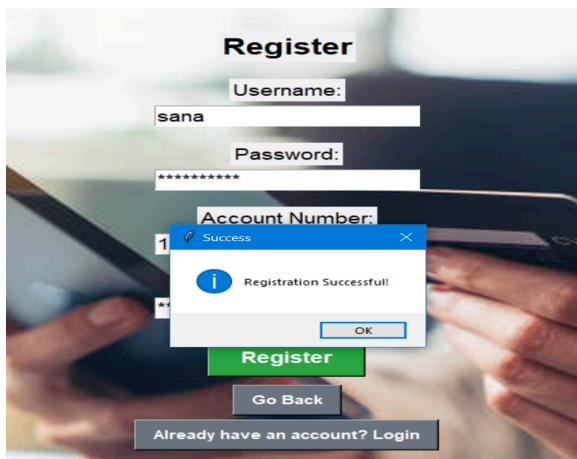
B6:



B4:



B7:



Branch Coverage:

- Total Branches: 7
- Covered Branches by testcases: 7
- Coverage: $7/7 = 100\%$

2. Functionality: Login**Code :**

```
def login():
```

```
    global current_user
    username = login_username_entry.get().strip()
    password = login_password_entry.get().strip()
    account_number = login_account_number_entry.get().strip()
    pin = login_pin_entry.get().strip()
```

```
    B1:    if username in users and users[username]["password"] == password and
users[username]["account_number"] == account_number and users[username]["pin"] == pin:
```

```
        current_user = username
        messagebox.showinfo("Login", "Login Successful")
        switch_frame/dashboard_frame)
        update_dashboard()
```

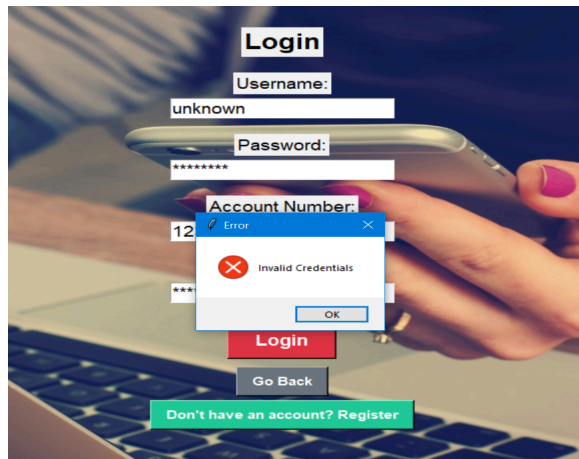
```
    B2: else:
        messagebox.showerror("Error", "Invalid Credentials")
```

Summary Report :

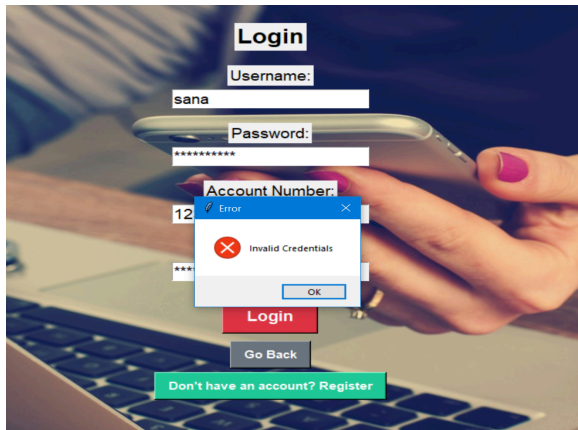
Test Case ID	Test Case Description	Test Steps	Test Data	Expected Result	Actual Result	Status	Branch Covered
TC_L01	Invalid Username	1. Go to Login Page	Username: wrong,	Error: Invalid credentials	Error: Invalid credentials	Pass	B2
		2. Enter wrong username	Password: pass1234, Account: 1234567890,				
		3. Click Login	PIN: 1234				
TC_L02	Invalid Password	1. Go to Login Page	Username: user1,	Error: Invalid credentials	Error: Invalid credentials	Pass	B2
		2. Enter wrong password	Password: 123, Account: 1234567890,				
		3. Click Login	PIN: 1234				
TC_L03	Invalid Account Number	1. Go to Login Page	Username: user1,	Error: Invalid credentials	Error: Invalid credentials	Pass	B2
		2. Enter wrong account number	Password: pass1234, Account: 0000000000,				
		3. Click Login	PIN: 1234				
TC_L04	Invalid PIN	1. Go to Login Page	Username: user1,	Error: Invalid credentials	Error: Invalid credentials	Pass	B2
		2. Enter wrong PIN	Password: pass1234, Account: 1234567890,				
		3. Click Login	PIN: 0000				
TC_L05	Valid Login	1. Go to Login Page	Username: user1,	Login successful	Login successful	Pass	B1
		2. Enter valid credentials	Password: pass1234, Account: 1234567890,				
		3. Click Login	PIN: 1234				

Test Case

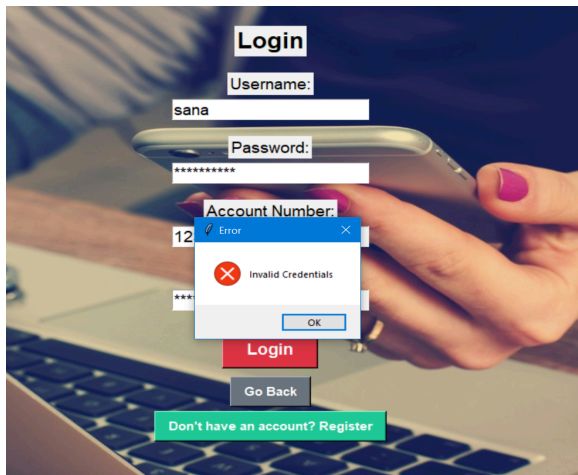
B2:



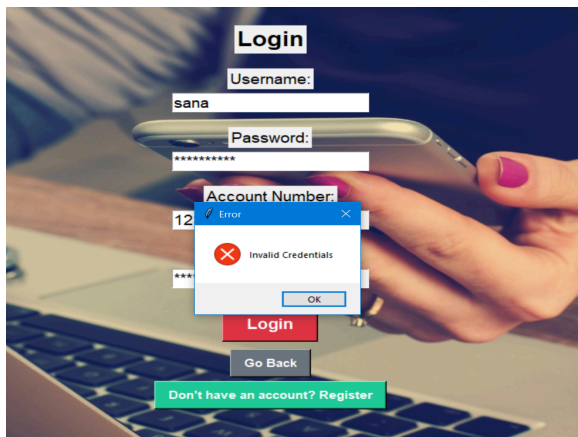
B1:



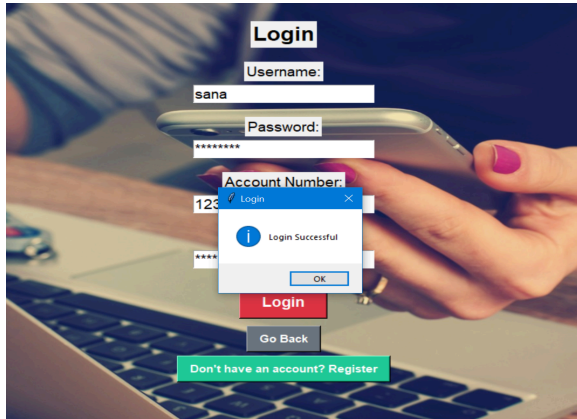
B2:



B1:



TC-LO5



Branch Coverage:

- Total Branches: 2
- Covered Branches by testcases: 2
- Coverage: $2/2 = 100\%$

3. Functionality: Deposit

Code:

```
def deposit():  
    amount = deposit_entry.get().strip()
```

B1: if not amount.isdigit() or int(amount) <= 0:

```
    messagebox.showerror("Error", "Enter a valid positive amount!")  
    return
```

B2: if int(amount) > 100000:

```
    messagebox.showerror("Error", "Deposit amount exceeds the limit of $100,000!")  
    return
```

B3: if amount valid and $\leq 100,000$:

```
    users[current_user]["balance"] += int(amount)  
    messagebox.showinfo("Success", f"Deposited ${amount} successfully!")
```

```

update_dashboard()
deposit_entry.delete(0, tk.END)

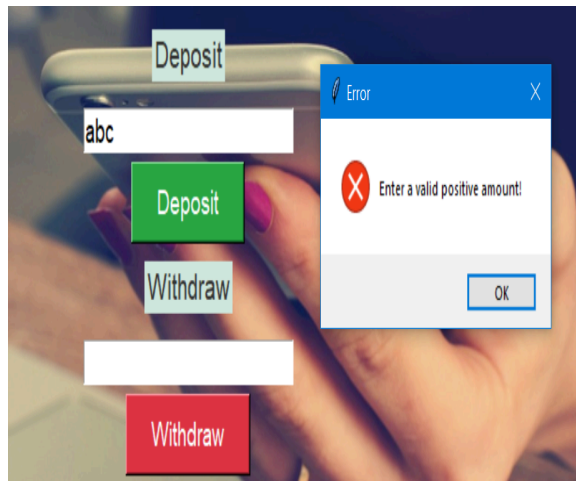
```

Summary Report:

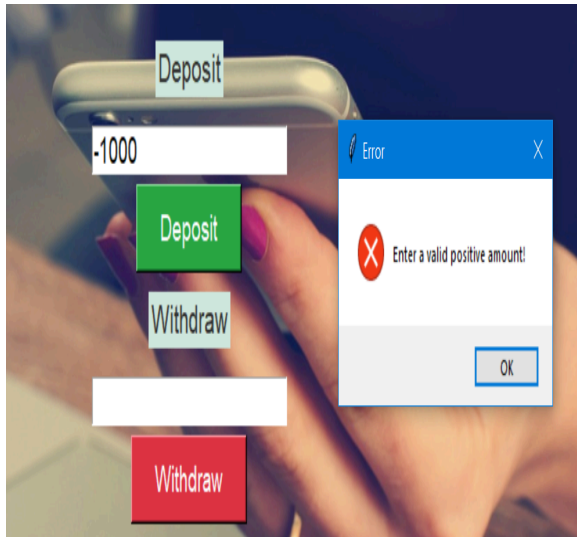
Test Case ID	Test Case Description	Test Steps	Test Data	Expected Result	Actual Result	Branch Covered	Status
TC_D01	Empty Amount Field	1. Log in as user1	Amount: (empty)	Error: Enter valid amount	Error: Enter valid positive amount	B1	Pass
		2. Leave amount empty					
		3. Click Deposit					
TC_D02	Negative Amount	1. Log in as user1	Amount: -500	Error: Enter valid positive amount	Error: Enter valid positive amount	B1	Pass
		2. Enter -500					
		3. Click Deposit					
TC_D03	Exceeding Deposit Limit	1. Log in as user1	Amount: 150000	Error: Deposit amount exceeds limit	Error: Deposit amount exceeds limit	B2	Pass
		2. Enter 150000					
		3. Click Deposit					
TC_D04	Valid Amount	1. Log in as user1	Amount: 5000	Deposit successful, balance updated	Deposit successful, balance updated	B3	Pass
		2. Enter 5000					
		3. Click Deposit					

Test Case

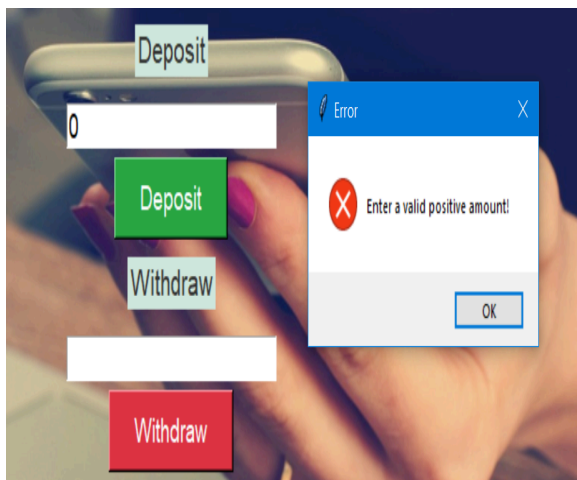
TC_D01



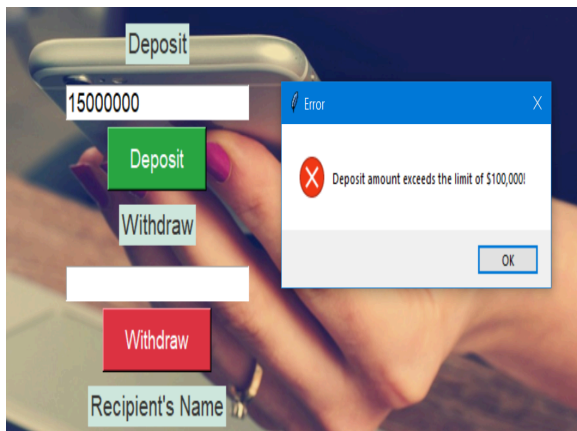
TC_D02



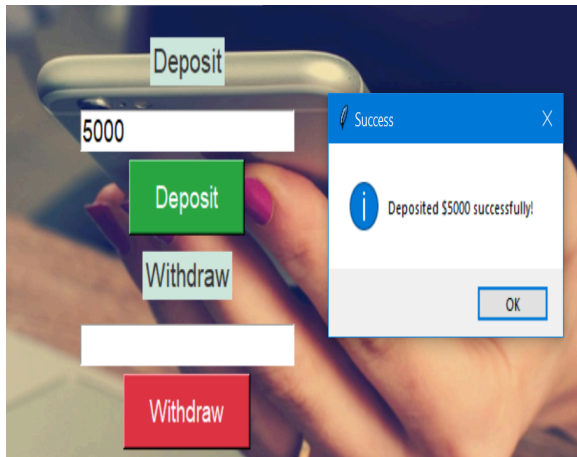
TC_D03



TC_D04



TC_D05



Branch Coverage:

- Total Branches: 3
- Covered Branches by testcases: 3
- Coverage: $3/3 = 100\%$

4. Functionality: Withdraw

Code :

```
def withdraw():  
    amount = withdraw_entry.get().strip()
```

```
B1: if not amount.isdigit() or int(amount) <= 0:  
    messagebox.showerror("Error", "Enter a valid positive amount!")  
    return  
    amount = int(amount)
```

```
B2: if users[current_user]['balance'] >= amount:  
    if amount > 50000:  
        messagebox.showerror("Error", "Daily withdrawal limit exceeded ($50,000 max)!")  
        return
```

```
B3: if users[current_user]['balance'] < amount:
```



```
messagebox.showerror("Error", "Insufficient Balance!")
```

B4: else:

```
users[current_user]['balance'] -= amount
```

```
messagebox.showinfo("Success", "Withdrawal Successful!")
```

```
update_dashboard()
```

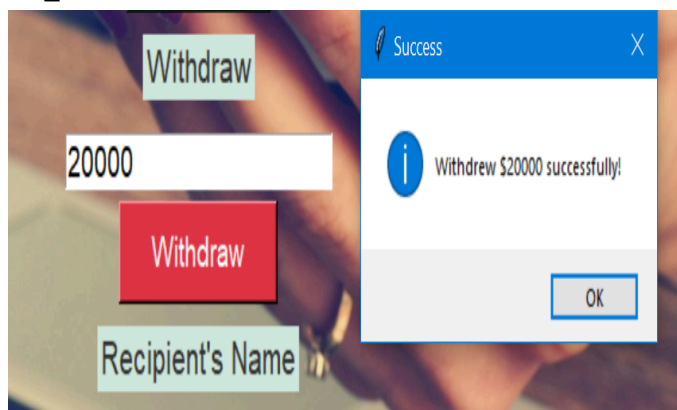
```
withdraw_entry.delete(0, tk.END)
```

Summary Report:

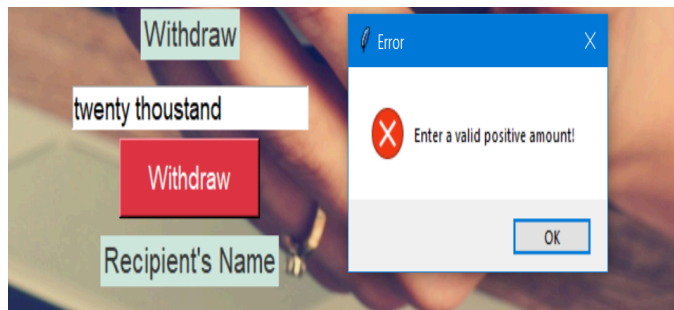
Test Case ID	Test Case Description	Test Steps	Test Data	Expected Result	Actual Result	Branch Covered	Status
TC_WQ_01	Amount is not numeric	1. Log in	Amount: "abc"	Error: Enter a valid positive amount!	Error: Enter a valid positive amount!	B1	Pass
		2. Enter "abc" in amount field					
		3. Click Withdraw					
TC_WQ_02	Amount is zero	1. Log in	Amount: 0	Error: Enter a valid positive amount!	Error: Enter a valid positive amount!	B1	Pass
		2. Enter 0					
		3. Click Withdraw					
TC_WQ_03	Amount is negative	1. Log in	Amount: -100	Error: Enter a valid positive amount!	Error: Enter a valid positive amount!	B1	Pass
		2. Enter -100					
		3. Click Withdraw					
TC_WQ_04	Amount > balance	1. Log in	Amount: 100000, Balance: 50000	Error: Insufficient Balance!	Error: Insufficient Balance!	B3	Pass
		2. Enter 100000					Pass
		3. User's balance is 50000					
		4. Click Withdraw					
TC_WQ_05	Amount ≤ balance but > 50000 (daily limit exceeded)	1. Log in	Amount: 60000, Balance: 100000	Error: Daily withdrawal limit exceeded (\$50,000 max)!	Error: Daily withdrawal limit exceeded (\$50,000 max)!	B2	Pass
		2. Enter 60000					Pass
		3. User's balance is 100000					
		4. Click Withdraw					
TC_WQ_06	Valid withdrawal (≤ balance and ≤ 50000)	1. Log in 2. Enter 20000	Amount: 20000, Balance: 50000	Withdrawal Successful!	Withdrawal Successful!	B4	Pass
		3. User's balance is 50000					
		4. Click Withdraw					

Test Case

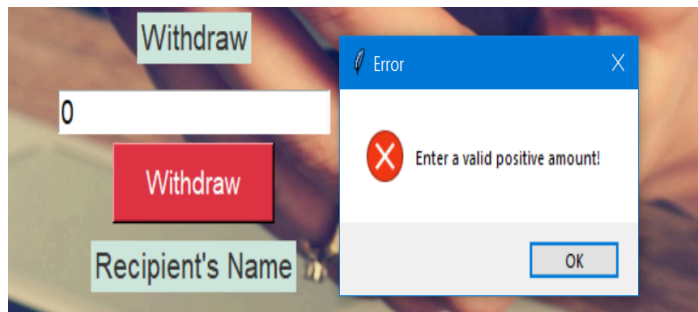
TC_W01



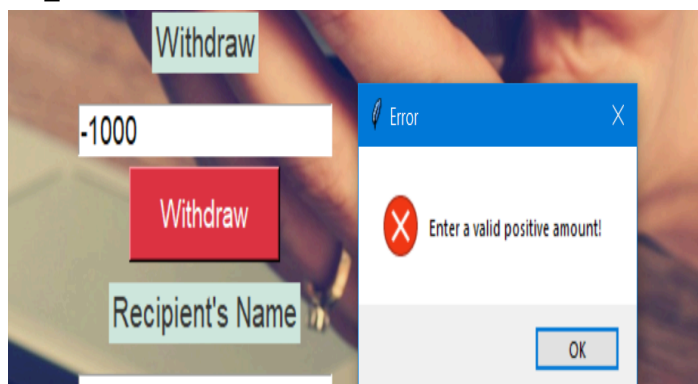
TC_W02



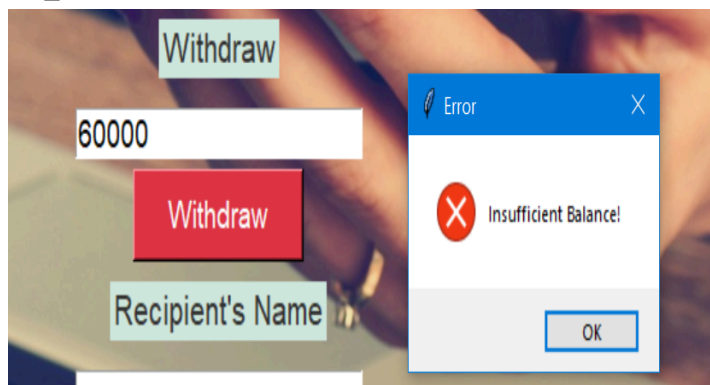
TC_W03



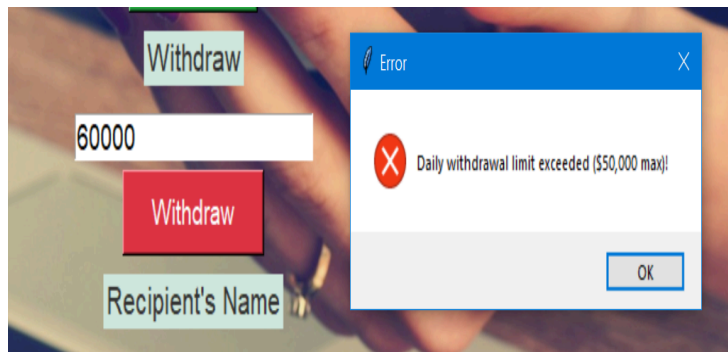
TC_W04



TC_W05



TC_W06



Branch Coverage:

- Total Branches: 4
- Covered Branches by testcases: 4
- Coverage: $4/4 = 100\%$

5. Functionality: Transfer

Code :

def transfer():

```
    recipient = transfer_recipient_entry.get().strip()
    amount = transfer_amount_entry.get().strip()
```

B1: if recipient == current_user:

```
    messagebox.showerror("Error", "Cannot transfer to self!")
    return
```

B2: if recipient not in users:

```
    messagebox.showerror("Error", "Recipient not found!")
    return
```

B3: if not amount.isdigit() or int(amount) <= 0:

```
    messagebox.showerror("Error", "Enter a valid positive amount!")
    return
```

B4: if int(amount) > users[current_user]['balance']:

```
    messagebox.showerror("Error", "Limit Crossed!")
    return
```

```

B5: else  users[current_user]['balance'] -= int(amount)
           users[recipient]['balance'] += int(amount)
           messagebox.showinfo("Success", f"Transferred ${amount} to {recipient} successfully!")
           update_dashboard()
           transfer_recipient_entry.delete(0, tk.END)
           transfer_amount_entry.delete(0, tk.END)

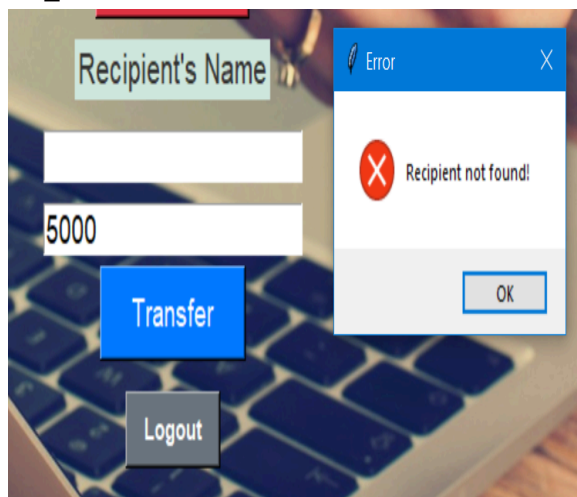
```

Summary Report:

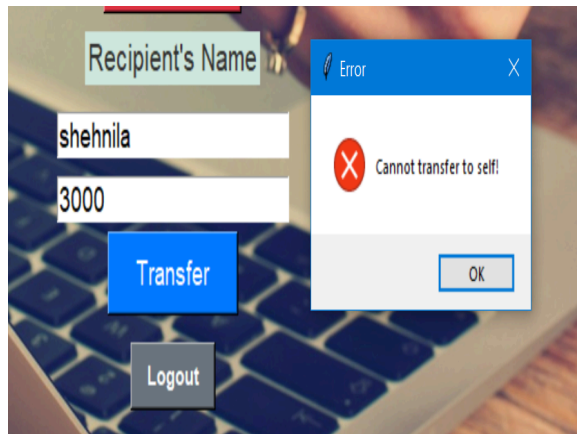
Test Case ID	Test Case Description	Test Steps	Test Data	Expected Result	Actual Result	Branch Covered	Status
TC_T01	Transfer to Self Not Allowed	1. Log in as current_user 2. Enter current_user as recipient 3. Enter valid amount 4. Click Transfer	To User: current_user Amount: 1000	Error: Cannot transfer to self!	Error: Cannot transfer to self!	B1	Pass
TC_T02	Recipient Not Found	1. Log in as current_user 2. Enter a non-existing user 3. Enter valid amount 4. Click Transfer	To User: unknown_user Amount: 500	Error: Recipient not found!	Error: Recipient not found!	B2	Pass
TC_T03	Invalid Amount (Non-digit Input)	1. Log in as current_user 2. Enter valid recipient 3. Enter non-digit amount like "five hundred" 4. Click Transfer	To User: user2 Amount: "five hundred"	Error: Enter a valid positive amount!	Error: Enter a valid positive amount!	B3	Pass
TC_T04	Invalid Amount (Zero or Negative)	1. Log in as current_user 2. Enter valid recipient 3. Enter amount as 0 or negative value 4. Click Transfer	To User: user2 Amount: 0 or -100	Error: Enter a valid positive amount!	Error: Enter a valid positive amount!	B3	Pass
TC_T05	Insufficient Balance (Limit Crossed)	1. Log in as current_user 2. Enter valid recipient 3. Enter amount greater than balance 4. Click Transfer	To User: user2 Amount: 100000 (balance=5000)	Error: Limit Crossed!	Error: Limit Crossed!	B4	Pass
TC_T06	Successful Transfer	1. Log in as current_user 2. Enter valid recipient 3. Enter amount less than or equal to balance 4. Click Transfer	To User: user2 Amount: 3000 (balance=5000)	Success: Transferred \$3000 to user2 successfully!	Success: Transferred \$3000 to user2 successfully!	B5	Pass

Test Case

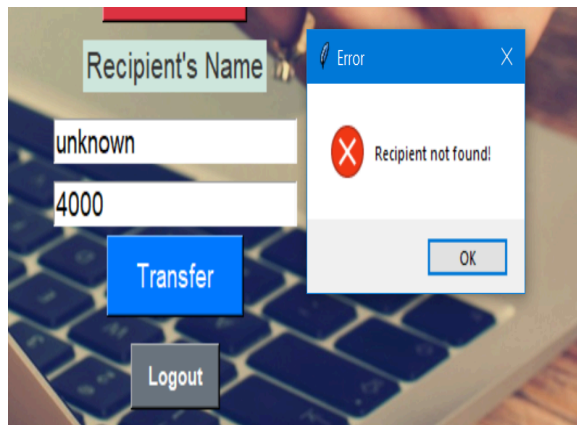
TC_01



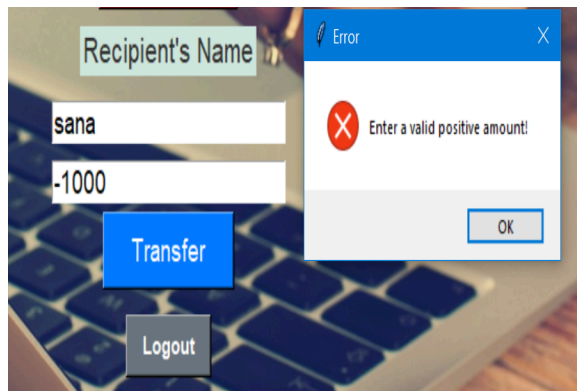
TC_02



TC_03

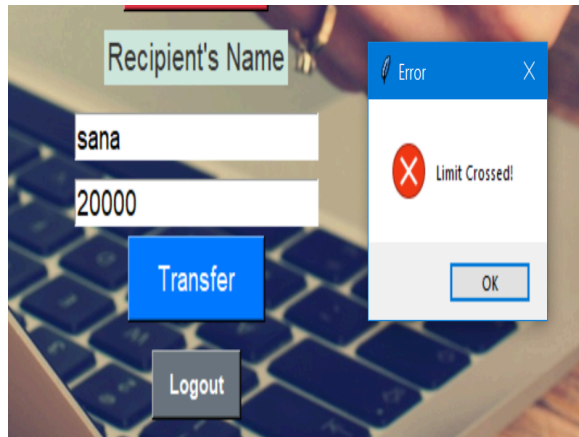


TC_04

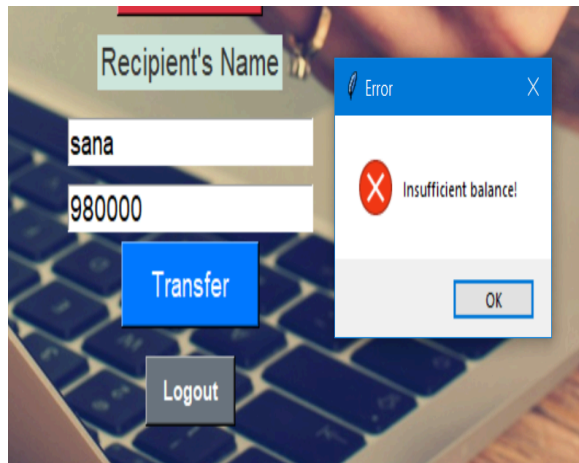


TC_05

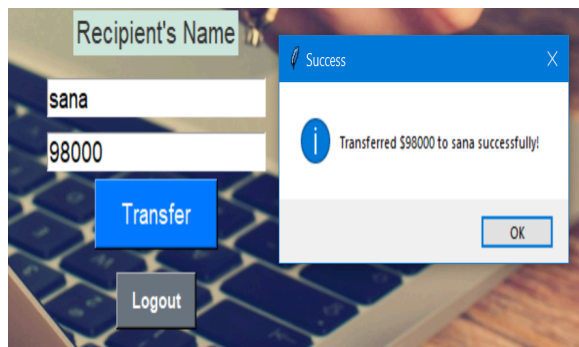
TC_06



TC_07



TC_08



Branch Coverage:

- Total Branches: 5
- Covered Branches : 5
- Coverage: $4/4 = 100\%$

Final Summary Table

Summary Table			
Functionality	Total Branches	Covered Branches	Branch Coverage
Registartion	7	7	100%
Login	2	2	100%
Deposit	3	3	100%
Withdraw	4	4	100%
Transfer	5	5	100%

i. Draw a comparison of selected technique with other types of White Box Testing techniques

White Box Testing techniques can be divided into Statement Coverage, Branch Coverage, Path Coverage, Condition Coverage and Loop Testing, but in our project we have chosen **Branch Coverage** as the main technique. What branch coverage means is that during testing all the possible branches (i. e. each decision point 's true and false condition) are executed at least once by us. Compared to Statement Coverage, which only checks whether each line of code is executed, branch coverage is more comprehensive because it validates all decisions, not just execution. Path Coverage also involves checking every possible execution path: this is rather exhaustive and it's impractical for large applications. Condition Coverage is focused on evaluating each Boolean sub-expression individually while Loop Testing deals with correctness of loops by checking for zero, one, and many iterations. Between them, branch coverage is a balanced approach between the most thorough of statements coverage and the least complex and

time-consuming of path covering. Therefore, for functional modules like registration, login, and transaction systems, it is an efficient and practical choice.

ii. Why did you choose this technique?

Branch Coverage was chosen as our White Box Test technique due to its great depth & practicality. Features like user registration, login, deposit, withdrawal, and fund transfer have multiple conditional checks & decision points where the check & answer must be valid (validate input formats, user uniqueness, boundary values, etc.), validate balance thresholds. Branch Coverage is perfect for this kind of logic based operations, because it checks whether all possible branches (true/false results of conditions) are executed during testing. It therefore helps us to make sure that the software behaves correctly in both normal & edge-case scenarios. It also allows us to measure the test coverage accurately to find where there are parts of code that have not been exercised in testing, thus improving overall code quality and reducing the risk of untested logical errors.

iii. On what level can we apply White Box Testing and when should we stop testing?

White box testing is most effective on unit testing and integration testing stages of the software development life cycle. During unit testing, the underlying logic and code of individual function and method is tested internal, i. e. register(input string) or login(input row) is tested from inside, but integration testing sees interactions between modules, especially where logical interaction takes place between different elements of the system. Because white box testing does not usually get used at the system or acceptance stage, we prefer using black box techniques when we want to test external behavior and user expectations of the system. Removing development from testing depends heavily on certain exit criteria. For example, testing should stop when all branch of code has been thoroughly tested, critical bugs have been fixed, test cases have been successfully passed, and the application fulfills predefined quality standards. Aside from these important decisions, the cost of time and cost constraints and the likelihood of finding new bugs in subsequent tests are other factors to consider. In short, no one can always test every scenario but one should find a balance between thoroughness and efficiency to get a reliable release.

